

Classifying Cars to Assist ALPR Systems

Sem Verkerk and Tyler Fabela

CS 171-02: Intro to Machine Learning

May 11, 2026

Introduction

Sometimes, it can be difficult to identify the make, model, and precise color of a vehicle when passing one by, but with photography, image processing, and powerful machine learning algorithms, this identification task can become quicker without any human error. And with the increasing number of cars on the road, the need for such algorithms has also grown, especially for applications such as law enforcement, traffic monitoring, and automated license plate recognition (ALPR) systems. Traditional ALPR systems primarily rely on license plate detection. However, these systems can often fail when license plates are obscured, damaged, or not visible. In cases where the license plate is unidentifiable, identifying the vehicle by its make, model, and color becomes essential. Recent improvements in deep learning make this possible, and we can create a convolutional neural network to achieve this goal.

Our objective was to train a convolutional neural network (CNN) with images of various vehicle makes and models such that the CNN can classify the make of a provided image of a vehicle. To do this, we first used a pre-trained model (CLIP) to improve the dataset quality. Then we built a robust CNN-based image classifier that would classify a dataset, which consisted of 10,000 images of cars. After our initial build, we tweaked the model to improve its accuracy.

The CNN classification model can help police or parking enforcement officers identify vehicles when they are on duty and need help identifying any vehicles that they

encounter on the job. The model can also help ALPR systems that are installed at intersections, where there are many cars that may be obscured and cannot be easily identified by current ALPR systems. This CNN model aims to support ALPR systems so that it can cut identification time and help monitor traffic.

Literature Review

There are projects and companies out there that are attempting to revolutionize the ALPR system. Some emerging trends in this market do include the inclusion of AI or Deep Learning models to assist in identifying cars. Particularly, MCA is attempting to build new ALPR technology with MMCR (Make, Model, Color, and Recognition) ideology, which is similar to our aspirations. Our innovation aims to address the dense workload of ALPR systems installed at traffic stops by introducing stored memory to the ALPR system. Stored memory gives lots of different possibilities, all of which can assist law enforcement in the area.

Deep learning has become the dominant approach when it comes to image classification, which is why we built a CNN model to identify the make of cars given an image. Key concepts for this project include:

- Convolutional Neural Networks (CNN): used for extracting spatial features from images
- Data Augmentation: improving generalization by artificially increasing dataset diversity

- Automated License Plate Recognition (ALPR): this is a system currently used by the police to identify cars' license plates
- Car "make": the brand or manufacturer that produced the vehicle

Methodology

The dataset we used was from the Kaggle website and is called "60,000+ Images of Cars," which contained over 60,000 images of cars, both interior and exterior. However, we cleaned and shrunk the dataset to only 10,000 images of the exterior of the car by removing interior images and duplicate images. The model uses a single classification head for each image input, where the output will be about the car's make.

To pre-process, we used a pre-trained model called CLIP. First, we extracted the 60,000+ images in the dataset. We used the CLIP model to compute similarity scores between the images and the text prompts. To clean the dataset, we used three text prompts to score exterior images, interior images, and junk images (close-ups, logos, not full-body pictures, etc). To compute the quality score, the model used this formula:

$$score = S_{ext} - \max(S_{int}, S_{junk})$$

We filtered images using a threshold: $score > -0.05$. Finally, we balanced the dataset by limiting it to around 250 cars per make and around 37 makes total.

As for feature engineering, we resized the images to fit a uniform format of 192 x 192 pixels. We also rescaled the pixel values (normalization) and then augmented the data (rotated, zoomed, altered brightness, and flipped the images as necessary) to ensure that the images were consistent for our model.

To achieve our goals, we designed a custom CNN model for classification. The network consists of five convolutional blocks with the filters increasing progressively (32 → 64 → 128 → 256 → 512). Each of these blocks includes convolution (3x3 kernels, ReLu activation), batch normalization, max pooling, and dropout (in the deeper layers). To prevent overfitting, we made sure our model had a dropout layer (0.5), L2 weight regularization, and batch normalization. Instead of flattening, which is usually seen in CNNs, our model uses Global Average Pooling (GAP) to reduce parameters and improve generalization.

While we originally had planned to classify the make, model, color, and year of the car, we decided to simplify the CNN to only classify the make of the car. Our singular classification head has a dense layer of around 512 units, with a dropout of 0.5, and an output layer that predicts probability distribution over car makes.

As for our training strategy, we used Categorical Cross-Entropy with Label Smoothing (0.1) to quantify the difference between the model's prediction and the actual make of the car. As for our optimizer, we used Adam with a learning rate of 0.001.

To further address any imbalance, we weighted the classes using this formula:

$$w_c = \frac{N}{|C| \cdot n_c}$$

Where N are the total samples, and n_c are the samples in class c .

Our model also used callbacks:

- EarlyStopping: stopped training if the validation loss didn't improve
- ReduceLROnPlateau: lowered the learning rate when progress stalled
- ModelCheckpoint: saved the best model

We evaluated the model on the following metrics:

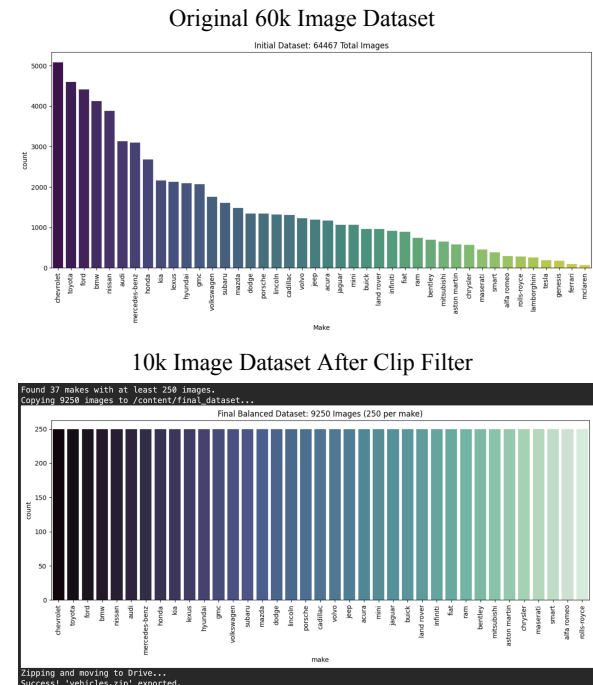
- Accuracy
- Precision
- Recall
- F1-score

Implementation

We primarily used Google Colab to run our program. We split our code into three primary sections: (1) data filtering, (2) training and evaluating our custom CNN, and (3) training and evaluating a pretrained model to compare against our custom CNN.

For the data filtering, we started off with over 60,000 images of vehicles from Kaggle, including car interiors and exteriors, and we needed to reduce the data so that there were only up to 10,000 images of exterior images to train on only. We used a CLIP filter from PyTorch in order to rank

each of the 60,000 images for how accurately they represented a car's exterior and picked the top 250 images for each make of car, resulting in 37 makes and 250 images each for a total of 9,250 images to train on. For each of the three main processes, we would take zip files located in carefully placed and named folders in Google Drive to use the images (for this step, we zipped up the 9,250 images and put them into the Google Drive Folder under a name for the custom and pretrained CNN to use).



Next, we implemented the TensorFlow custom CNN, which uses an 80% training, 10% validation, and 10% testing split. In order to expand our dataset, we used various forms of augmentation, which include image rotation, image stretching, image flipping, and color alterations. Next, we used five convolutional layers with doubling batch sizes for each layer, starting at 32, and a dense layer of batch size 512 (we had

various iterations to try resolving underfitting and overfitting, so we needed to play around with components such as L2 regularization, dropout, batch sizes, and image sizes). For the compilation of the model, we ended up using the “Adam” optimizer and cross-entropy, configuring it as we needed. Then we would run the model on 50 epochs with configured callbacks (See “Experimental Setup” Section for specifics).

```
# Training
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=50,
    callbacks=callbacks,
    class_weight=class_weight_dict
)
```

Finally, we used MobileNetV2 to perform transfer learning for our pretrained model. This model uses transfer learning from the MobileNetV2 CNN in order to recognize the features of the images we use, but it requires us to first train by freezing layers to prevent the model from changing its layers, then unfreezing them for the model to slowly learn the dataset properly.

Experimental Setup

As stated earlier, we used a data split of 80% training, 10% validation, and 10% testing. Essentially, we train the model using the 9,250 image data and augment the data with the settings in the screenshot below:

```
# Training Augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.15,
    height_shift_range=0.15,
    zoom_range=0.15,
    horizontal_flip=True,
    fill_mode='nearest',
    shear_range=0.1,
    brightness_range=[0.75, 1.25]
)

# Validation Augmentation
val_test_datagen = ImageDataGenerator(rescale=1./255)
```

This will help prevent the model from overfitting, which was one of the issues we faced.

For the layers of the custom CNN, we chose to have 5 convolutional layers, each having “relu” activations, a 3x3 filter, and the “same” padding. The topmost layer starts with a batch of 32, the subsequent one gets a batch of 64, and we continue to double the batch amount until we get to 512 for the 5th one. After each of these layers, we apply batch normalization and a max pooling with a matrix size of 2x2. Next, we have the dense layer with batch size 512, applying a global average pooling before entering this layer. We gave the dense layer the hyperparameters “activation='relu' and kernel_regularizer=regularizers.l2(0.0001)”. We chose this specific setup because adding any additional layers or any L2 regularization caused the model to learn the training data well but overfit in the end, whereas reducing the number of layers or adding any additional dropout made the model too simple and incomplete to get the training accuracy up.

```
def build_custom_cnn():
    inputs = Input(shape=(img_size[0], img_size[1], 3))

    # 5 Convolutional Layers
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = Conv2D(256, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = Conv2D(512, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)

    x = GlobalAveragePooling2D()(x)

    # 1 Dense Layer
    x = Dense(512, activation='relu', kernel_regularizer=regularizers.l2(0.0001))(x)
    x = Dropout(0.5)(x)
    outputs = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs=inputs, outputs=outputs)
    return model

model = build_custom_cnn()
```

For our compilation hyperparameters of the model, we used a batch size of 64, 50 epochs, a learning rate of 0.001, and, as previously stated, the categorical cross-entropy with label smoothing (0.1).

In the end, we ended up with the number of parameters in the screenshot below:

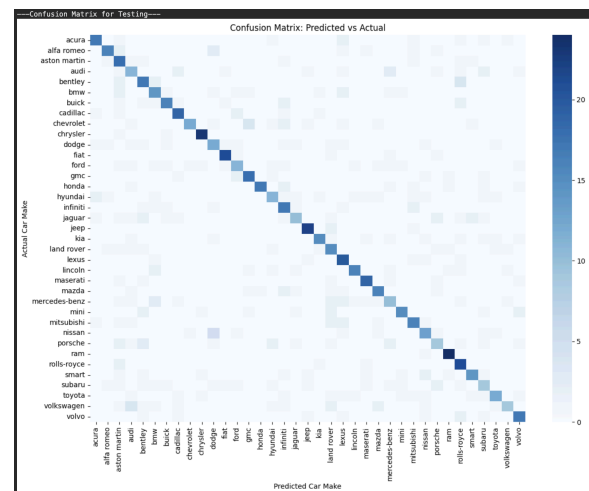
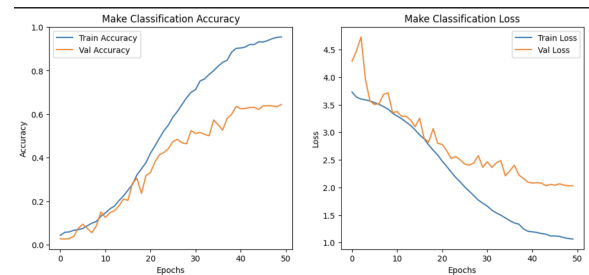
Total params: 1,854,181 (7.07 MB)
Trainable params: 1,852,197 (7.07 MB)
Non-trainable params: 1,984 (7.75 KB)

Results

For this project, we decided to evaluate the models on the metrics mentioned in the **Methodology** section. To expand on those metrics:

- Accuracy is a good metric to use when the dataset is balanced and is a good way to display overall correctness. In this case, out of all the predictions, accuracy depicts how many were correct.

- Precision measures how often predictions for a given class are correct. In the case of our model, this would be if the model correctly predicted 120 Toyotas and only got 90 of those correct, then the model would have 75% precision for that make.
- Recall focuses on the true positive predictions over the true positive added to the false negatives. This would be the number of true positive predicted Toyotas over the total actual number of Toyotas.
- F1 Score is the harmonic mean of precision and recall.

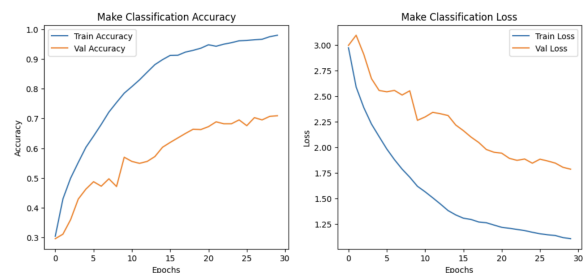


	precision	recall	f1-score	support
acura	0.82	0.72	0.77	25
alfa romeo	0.59	0.64	0.62	25
aston martin	0.57	0.64	0.60	25
audi	0.38	0.48	0.42	25
bentley	0.78	0.72	0.75	25
bmw	0.41	0.56	0.47	25
buick	0.74	0.68	0.71	25
cadillac	0.69	0.88	0.77	25
chevrolet	0.64	0.56	0.60	25
chrysler	0.65	0.80	0.71	25
dodge	0.62	0.60	0.61	25
fiat	0.76	0.76	0.76	25
ford	0.42	0.52	0.46	25
gmc	0.74	0.68	0.71	25
honda	0.68	0.84	0.75	25
hyundai	0.38	0.44	0.41	25
infiniti	0.68	0.60	0.64	25
jaguar	0.43	0.40	0.42	25
jeep	0.79	0.88	0.83	25
kia	0.61	0.56	0.58	25
land rover	0.54	0.60	0.57	25
lexus	0.53	0.72	0.61	25
lincoln	0.70	0.64	0.67	25
maserati	0.78	0.56	0.65	25
mazda	0.74	0.56	0.64	25
mercedes-benz	0.32	0.36	0.34	25
mini	0.82	0.56	0.67	25
mitsubishi	0.71	0.48	0.57	25
nissan	0.37	0.44	0.40	25
porsche	0.60	0.36	0.45	25
ram	0.83	1.00	0.91	25
rolls-royce	0.69	0.80	0.74	25
smart	0.95	0.72	0.82	25
subaru	0.46	0.44	0.45	25
toyota	0.67	0.48	0.56	25
volkswagen	0.73	0.44	0.55	25
volvo	0.61	0.68	0.64	25

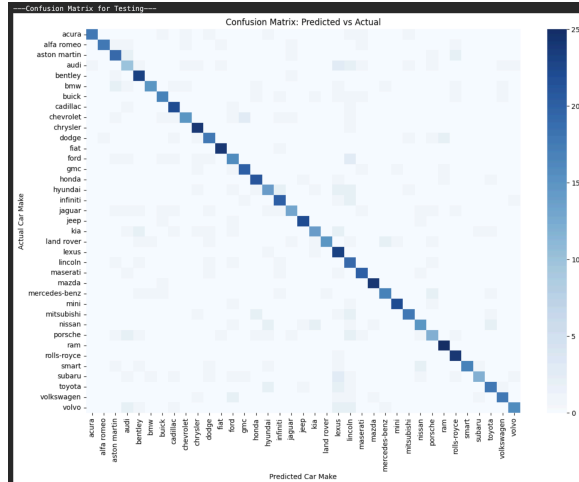
From above, we get to see that the model can become very good at classifying certain car makes, even on images it hasn't worked with before. Even if for a lot of the car makes, it has odds similar to or slightly better than a coin flip, it is still performing significantly better than a random guess with odds of $1/37 \approx 3\%$. However, we understand that with more time, data, and computing power, we could bring up these odds so that the model can perform with very high accuracy, matching closely with pretrained and highly trained models.

In comparison to the pretrained model, our custom CNN did—in fact—perform worse, but it isn't as terribly off as we would have anticipated. Of course, we needed to refine the pretrained model so it doesn't overfit

(again with more time, data, and computing power), but the screenshots below still demonstrate that a pretrained model can perform well in comparison to one that was constructed from scratch. To reiterate, this pretrained model got a training/validation/testing accuracy of about 95%/70%/70%, whereas the custom model was worse with a training/validation/testing accuracy of about 90%/60%/60%



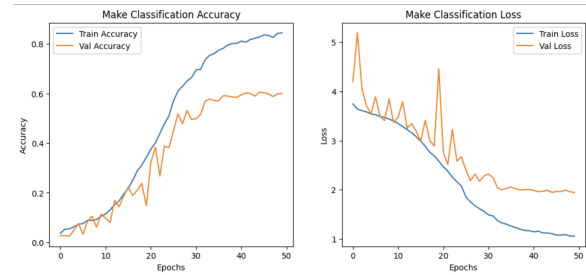
--- Classification Report ---				
	precision	recall	f1-score	support
acura	0.95	0.72	0.82	25
alfa romeo	0.95	0.72	0.82	25
aston martin	0.66	0.76	0.70	25
audi	0.42	0.40	0.41	25
bentley	0.70	0.92	0.79	25
bmw	0.83	0.60	0.70	25
buick	0.71	0.68	0.69	25
cadillac	0.73	0.88	0.80	25
chevrolet	0.79	0.60	0.68	25
chrysler	0.75	0.96	0.84	25
dodge	0.67	0.72	0.69	25
fiat	0.89	0.96	0.92	25
ford	0.57	0.64	0.60	25
gmc	0.80	0.80	0.80	25
honda	0.78	0.84	0.81	25
hyundai	0.61	0.56	0.58	25
infiniti	0.69	0.80	0.74	25
jaguar	0.68	0.52	0.59	25
jeep	0.88	0.88	0.88	25
kia	0.70	0.56	0.62	25
land rover	0.94	0.60	0.73	25
lexus	0.50	0.92	0.65	25
lincoln	0.42	0.76	0.54	25
maserati	0.74	0.80	0.77	25
mazda	0.92	0.96	0.94	25
mercedes-benz	0.81	0.68	0.74	25
mini	0.92	0.88	0.90	25
mitsubishi	0.82	0.72	0.77	25
nissan	0.68	0.60	0.64	25
porsche	0.57	0.48	0.52	25
ram	0.86	1.00	0.93	25
rolls-royce	0.75	0.96	0.84	25
smart	1.00	0.68	0.81	25
subaru	0.80	0.48	0.60	25
toyota	0.75	0.72	0.73	25
volkswagen	0.82	0.72	0.77	25
volvo	0.84	0.64	0.73	25
accuracy			0.73	925
macro avg	0.75	0.73	0.73	925
weighted avg	0.75	0.73	0.73	925



Discussion

The most challenging part of this task was to find the correct balance between underfitting and overfitting for both the custom and pretrained models. Because our dataset was too small for having 37 different classes to categorize, our models would underfit in the sense that their training accuracies were typically below or at 50%, so the model wasn't able to properly recognize patterns. However, once we removed certain regularizations for the convolutional layers (i.e., lowering the dropout but removing L2 penalization), then the training accuracy would climb up to around 80%, but the validation and testing accuracy would stay between 55% and 60%. Because of how much higher this testing accuracy was compared to the underfitting model, we figured that the overfitting model (~80% training and ~60% testing accuracy) was the best version of what we could do, especially since it takes so long to adjust the hyperparameters and wait for the model to converge. We felt that it was best to use the overfitted model with the lowest loss, even if it meant sacrificing the highest testing

accuracy we could potentially get (other versions could read training/testing accuracies of +90%/65%).



Even though this was an issue early on, we would also say that trying to parse the data was a bit of a struggle, too. We had issues configuring the clip filter, resulting in data that wasn't as distributed or usable as we would have liked, which caused our various models to underfit. However, once we were able to verify that the data was balanced and of car exteriors, then our models started to improve their accuracy metrics by about 20%.

As said in the first challenge, the limitations we faced were the lack of computing power and the amount of time it takes to train a CNN. Each epoch takes between 60 and 100 seconds for the various models (including the finalized custom CNN and pretrained CNN), which ends up taking much longer with our epoch count, and we would have to run these models various times with small tweaks to the CNN's layers, improving the dataset, changing up the data augmentation, and altering the CNN's regularization methods. Because of how sensitive our custom and pretrained models ended up being, we were not able to find a balance between high accuracy and similar training

to testing accuracy—we were only able to get an underfitted model with odds as good as a coin flip or an overfitted model with 60% testing accuracy. In retrospect, we should have attempted to train the model with only 10 classes and 1,000 images per class, but we understand and have gained respect for the time and effort it takes to correctly assemble and train a CNN.

If we had more time and optimized computational resources, then we believe we could have increased how well these models perform for both training and validation/testing; and, with an easier task than having almost 40 different classes, then we could have achieved better results closer to 80% or 90% for both training and testing.

Conclusion

This project successfully developed a deep learning pipeline for vehicle make classification using a combination of CLIP-based preprocessing and a custom CNN model.

Although the model did not achieve state-of-the-art performance, on par with current revolutionary improvements being made to the ALPR system, we were able to successfully build a model that was able to identify the make of a car given an image. While this model cannot identify more than just the make, like we wanted it to do, this

work still contributes to improving intelligent vehicle recognition systems and can be extended to real-world applications such as smart surveillance and traffic monitoring. It can definitely be improved, first and foremost, by adding the other classification heads that we had originally intended to add. But also by giving the model a memory so that it can build upon itself and ultimately keep neighborhood traffic safe by alerting officers to inconsistencies in everyday traffic.

References

“Building a Smarter ALPR Program” Article:

<https://callmc.com/smarter-alpr-program-scale-scope/>

“The Future of ALPR: 7 Emerging Trends Revolutionizing Vehicle Recognition Technology” Article:

<https://www.platesmart.com/7-emerging-trends-vehicle-recognition-alpr/#:~:text=1.,through%20a%20single%20C%20unified%20interface.>

The Future of ALPR: 7 Emerging Trends Revolutionizing Vehicle Recognition Technology “60,000+ Images of Cars” Kaggle Dataset:

<https://www.kaggle.com/datasets/prodeau/the-car-connection-picture-dataset>